ANDROID DEVELOPMENT SERIES

Data Persistence & User Interaction

A comprehensive guide to SQLite, Room, Jetpack Compose Gestures, and Kotlin Flow.

PART 01

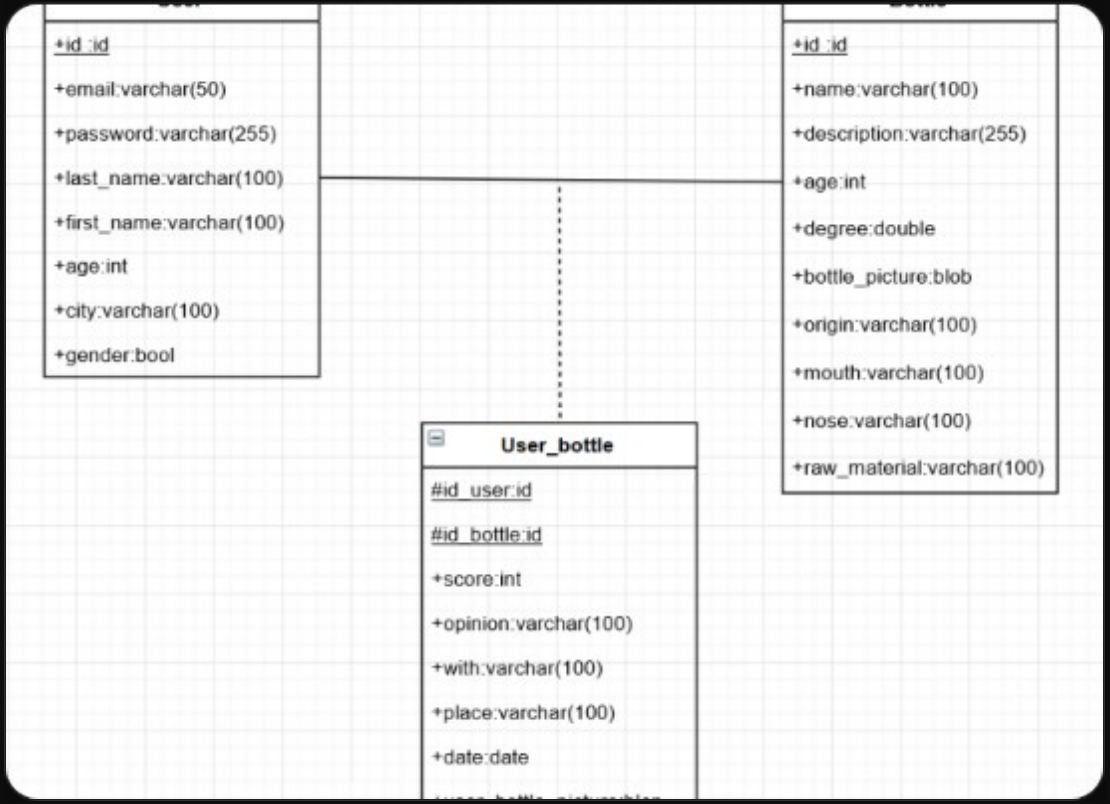
SQLite & Android Fundamentals

Understanding the bedrock of local data storage.

SQLite Architecture

The Embedded RDBMS

- **Serverless:** Unlike MySQL or Oracle, SQLite runs directly within the Android application process.
- **File-Based:** Data is stored in a simple file hierarchy.
- **Structured Data:** Utilizes standard tables, rows, and columns. Requires a Primary Key (conventionally _id) for unique row identification.
- **Lifecycle:** Critical for persisting game scores, user settings, and cache when the OS kills app processes to free memory.



Core SQLite Features

Embedded

Linked directly into the application as a library. No background server process is required, minimizing resource overhead on mobile devices.

SQL-92 Standard

Uses Structured Query Language (SQL) for data manipulation. Knowledge of SQL transfers easily from other database systems.

ADB Integration

Developers can interact directly with database files on emulators using the sqlite3 command-line tool via the ADB shell.

PART 02

The Room Persistence Library

Modernizing database access with Android Architecture Components.

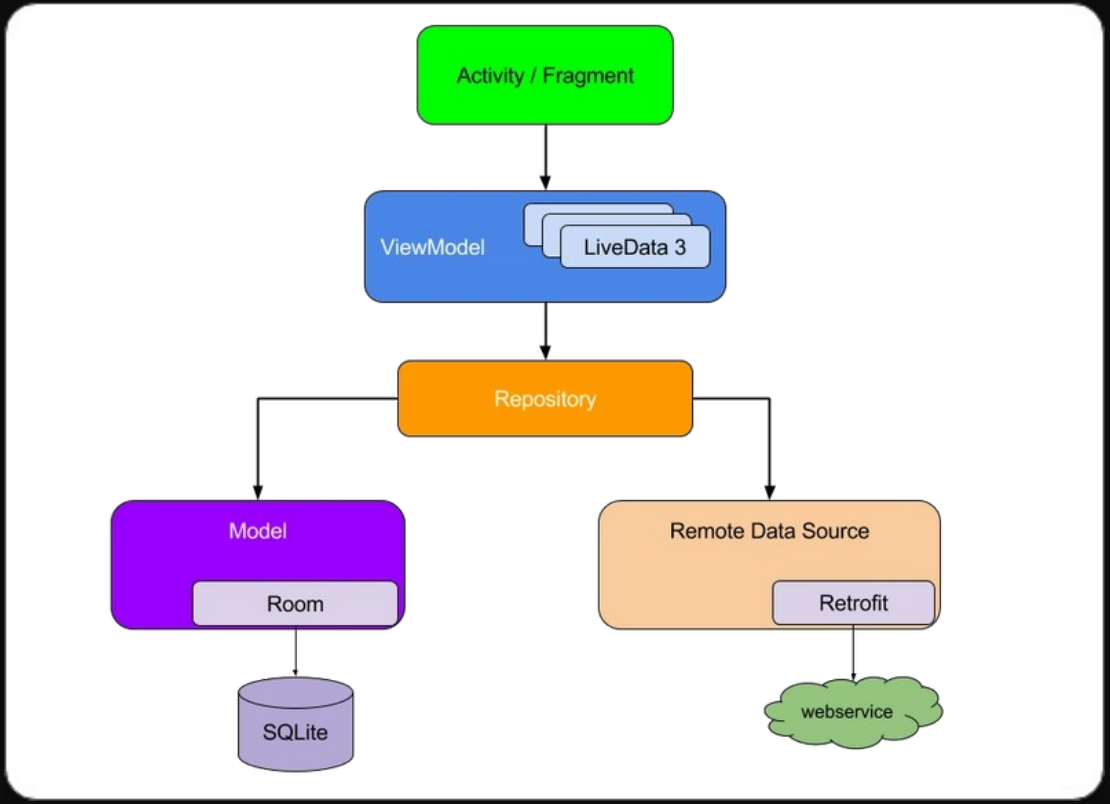
Room Architecture Pattern

Abstraction Layer

Room provides an abstraction layer over raw SQLite to allow fluent database access while harnessing the full power of SQLite.

Key Benefits

- **Compile-time Verification:** SQL queries are checked at compile time, preventing runtime crashes due to syntax errors.
- **Boilerplate Reduction:** Eliminates the need for verbose Cursor and ContentValues code.
- **LiveData/Flow Integration:** seamless observation of database changes.



The Three Pillars of Room

Entity

Represents a table within the database. Annotated with @Entity. Fields become columns; one field must be the @PrimaryKey.

DAO

Data Access Object. An interface annotated with @Dao containing methods to access the database (Insert, Update, Delete, Query).

Database

The abstract class extending RoomDatabase. It serves as the main access point and holds the database version and DAO definitions.

PART 03

Implementing Room

A step-by-step workflow for modern Android apps.

Development Workflow

Setup

Add KSP plugin and Room dependencies to Gradle.

Schema

Define @Entity data classes for tables.

Access

Create @Dao interfaces with SQL queries.

Repository

Isolate data sources from the UI logic.

UI

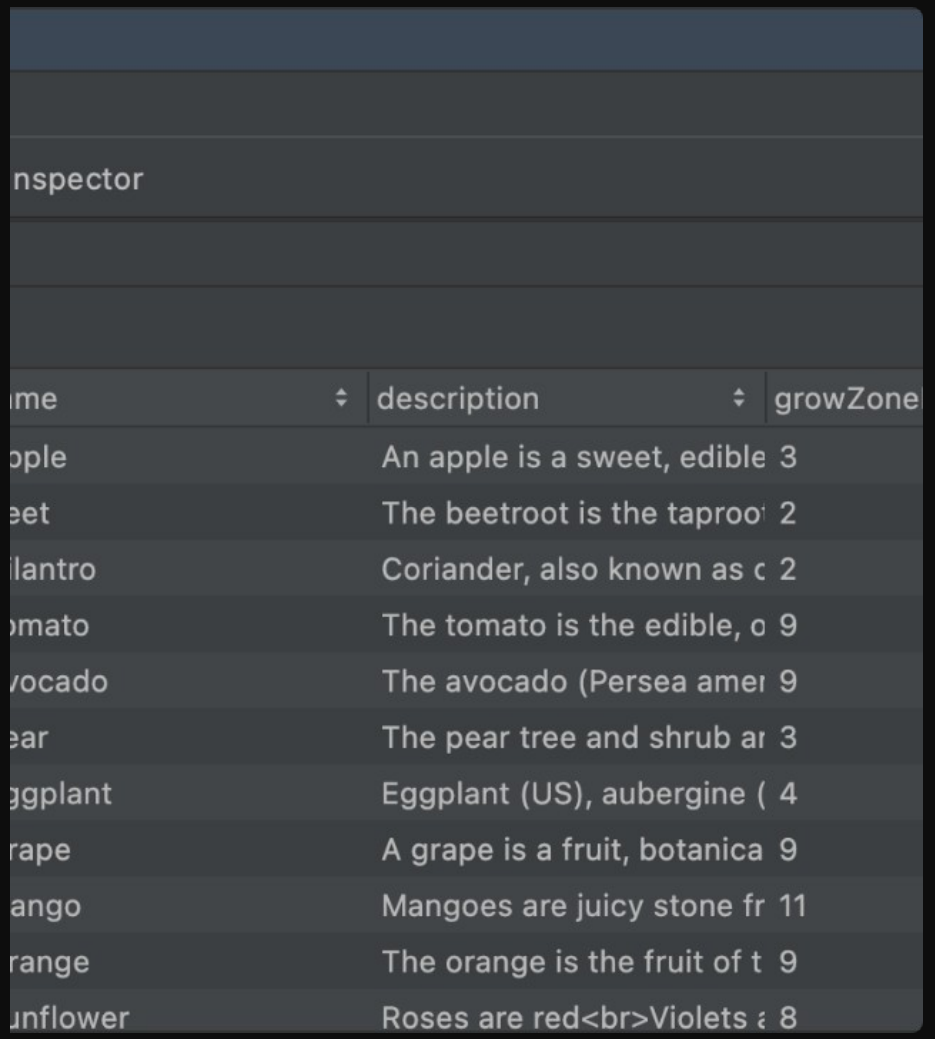
Connect ViewModel to Compose UI.

Database Inspector

Debugging databases is no longer a black box operation.

Included in Android Studio, the Database Inspector allows developers to:

- Query running databases in real-time.
- Modify values on the fly to test UI responses.
- Visualize schema relationships.
- Execute custom SQL commands against the live app.



PART 04

Gestures in Compose

Creating intuitive interactions beyond the simple click.

Primary Gesture Types

Tap & Press

Detected via detectTapGestures inside pointerInput. Supports single tap, double tap, and long press.

Drag & Scroll

draggable (1 axis) or detectDragGestures (2D). scrollable modifiers handle list physics.

Transform

Multi-touch operations like Pinch-to-Zoom, Pan, and Rotate handled by the transformable modifier.

Implementation Logic

Simple vs. Complex

Clickable Modifier: The simplest approach. Best for standard buttons or list items. Limited to single clicks.

```
Modifier.clickable { onClick() }
```

PointerInput Scope: Required for granular control. It suspends and awaits pointer events, allowing detection of distinct phases (down, move, up).

```
Modifier.pointerInput(Unit) {
detectTapGestures(...)
}
```

State Management

Gestures drive UI State. For example, a drag gesture updates an OffsetX and OffsetY state variable, which is then applied to the UI component using the .offset modifier.

For scrolling, rememberScrollState automatically manages the scroll position and physics.

PART 05

Kotlin Flow

Managing asynchronous data streams in a reactive environment.

The Stream Paradigm

Flow is built on the Producer-Intermediary-Consumer model.

Producers emit values over time. Intermediaries (operators like map, filter) modify the stream. Consumers collect the results.

Flow Type Comparison

Feature	Standard Flow	StateFlow	SharedFlow
Temperature	Cold (Lazy)	Hot (Active)	Hot (Active)
Subscribers	Single Consumer	Multiple Consumers	Multiple Consumers
Initial Value	None required	Mandatory	Optional
Use Case	DB queries, Network calls	UI State (ViewModel)	One-time Events (Snackbars)

Essential Operators

Transforming Data

Just like standard collection operators, map converts emitted values from one type to another (e.g., Entity to UI Model) before they reach the consumer.

Merging Streams

The zip operator combines values from two different flows. It waits for both flows to emit a value before pairing them, useful for combining results from parallel network requests.

Q&A

Discussion & Code Review